

## Festival

Nayra está en un festival jugando un juego donde el gran premio es un viaje a la Laguna Colorada. El juego consiste en usar tokens para comprar cupones. Comprar un cupón puede resultar en obtener tokens adicionales. El objetivo es obtener tantos cupones como sea posible.

Ella comienza el juego con  $A$  tokens. Hay  $N$  cupones, numerados desde 0 hasta  $N - 1$ . Nayra tiene que pagar  $P[i]$  tokens ( $0 \leq i < N$ ) para comprar el cupón  $i$  (y ella debe tener al menos  $P[i]$  tokens antes de la compra). Ella puede comprar cada cupón a lo más una vez.

Además, cada cupón  $i$  ( $0 \leq i < N$ ) tiene asignado un **tipo**, denotado por  $T[i]$ , que es un entero **entre 1 y 4, inclusive**. Después de que Nayra compra el cupón  $i$ , el número de tokens restantes es multiplicado por  $T[i]$ . Formalmente, si ella tiene  $X$  tokens en algún momento del juego, y compra el cupón  $i$  (que requiere que  $X \geq P[i]$ ), luego ella va a tener  $(X - P[i]) \cdot T[i]$  tokens después de la compra.

Su tarea es determinar los cupones que debe comprar Nayra y en qué orden, para maximizar el número total de **cupones** que tiene al final. Si hay más de una secuencia de compras mediante la cual obtiene ese máximo valor, puede reportar cualquiera de ellas.

## Detalles de Implementación

Debe implementar la siguiente función:

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- $A$ : la cantidad inicial de tokens que tiene Nayra.
- $P$ : un arreglo de longitud  $N$  especificando los precios de los cupones.
- $T$ : un arreglo de longitud  $N$  especificando los tipos de los cupones.
- Esta función es llamada exactamente una vez por cada caso de prueba.

Esta función debe retornar un arreglo  $R$ , que especifica las compras de Nayra de la siguiente manera:

- La longitud de  $R$  debe ser el máximo número de cupones que ella puede comprar.
- Los elementos del arreglo son los índices de los cupones que ella debe comprar, en orden cronológico. Es decir, ella primero compra el cupón  $R[0]$ , luego el cupón  $R[1]$ , y así

sucesivamente.

- Todos los elementos de  $R$  deben ser únicos.

Si no puede comprar ningún cupón,  $R$  debe ser un arreglo vacío.

## Restricciones

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$  para cada  $i$  tal que  $0 \leq i < N$ .
- $1 \leq T[i] \leq 4$  para cada  $i$  tal que  $0 \leq i < N$ .

## Subtareas

| Subtareas | Puntos | Restricciones Adicionales                                            |
|-----------|--------|----------------------------------------------------------------------|
| 1         | 5      | $T[i] = 1$ para cada $i$ tal que $0 \leq i < N$ .                    |
| 2         | 7      | $N \leq 3000$ ; $T[i] \leq 2$ para cada $i$ tal que $0 \leq i < N$ . |
| 3         | 12     | $T[i] \leq 2$ para cada $i$ tal que $0 \leq i < N$ .                 |
| 4         | 15     | $N \leq 70$                                                          |
| 5         | 27     | Nayra puede comprar los $N$ cupones (en algún orden).                |
| 6         | 16     | $(A - P[i]) \cdot T[i] < A$ para cada $i$ tal que $0 \leq i < N$ .   |
| 7         | 18     | Sin restricciones adicionales.                                       |

## Ejemplos

### Ejemplo 1

Considere la siguiente llamada.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Inicialmente, Nayra tiene  $A = 13$  tokens. Ella puede comprar 3 cupones en el orden mostrado a continuación:

| Cupón comprado | Precio del cupón | Tipo del cupón | Cantidad de tokens después de la compra |
|----------------|------------------|----------------|-----------------------------------------|
| 2              | 8                | 3              | $(13 - 8) \cdot 3 = 15$                 |
| 3              | 14               | 4              | $(15 - 14) \cdot 4 = 4$                 |
| 0              | 4                | 1              | $(4 - 4) \cdot 1 = 0$                   |

En este ejemplo, no es posible que Nayra compre más de 3 cupones, y la secuencia de compras descrita anteriormente es la única forma en que ella puede comprar 3 cupones. Por lo tanto, la función debe retornar  $[2, 3, 0]$ .

## Ejemplo 2

Considere la siguiente llamada.

```
max_coupons(9, [6, 5], [2, 3])
```

En este ejemplo, Nayra puede comprar ambos cupones en cualquier orden. Por lo tanto, la función debe retornar  $[0, 1]$  o  $[1, 0]$ .

## Ejemplo 3

Considere la siguiente llamada.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

En este ejemplo, Nayra tiene un token, que no es suficiente para comprar ningún cupón. Por lo tanto, la función debe retornar  $[]$  (un arreglo vacío).

## Evaluador de Ejemplo

Formato de entrada:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato de salida:

```
S  
R[0] R[1] ... R[S-1]
```

Aquí,  $S$  es la longitud del arreglo  $R$  retornado por `max_coupons`.